

Task 08 - Part 02: Question Answers (Q2 & Q3)

C# Fundamentals - Coding Against Abstraction

2. What do we mean by coding against an interface rather than a class? And what do we mean by coding against abstraction, not concreteness?

"Coding against an interface rather than a class" means that when you write code that depends on some other object, you declare the dependency using the **interface type** (e.g., `IVehicle`) rather than a specific, concrete class (e.g., `Car`). So instead of writing `Car myVehicle = new Car();` and working with it as a `Car` everywhere, you write `IVehicle myVehicle = new Car();` and only ever interact with it through the members the interface defines. The concrete class is still used to actually *create* the object, but everywhere else in the code only relies on the contract.

"Coding against abstraction, not concreteness" is the more general version of the same idea, and it applies to abstract classes too, not just interfaces. The principle is: write code that depends on the **most general, abstract type that still has everything the code actually needs** - an interface, an abstract class, or any base type - rather than depending on one specific, concrete implementation. This means your code doesn't care exactly which concrete class it's working with, only that the object fulfills a certain contract or behavior. The direct benefit is that the concrete implementation can be swapped, extended, or replaced later (a new class, a mock for testing, a different algorithm) without having to touch or break the code that depends on the abstraction - this is essentially the Dependency Inversion Principle (the "D" in SOLID).

3. What is abstraction as a guideline, and how can we implement this through what we've studied?

As a guideline, **abstraction** means focusing a design on *what* something does - the essential behavior and contract a piece of code needs to expose - while deliberately hiding *how* it actually does it. It's a way of managing complexity: consumers of a class or component should only need to understand its public contract, not its internal implementation details, so that implementation can change freely without breaking anything that depends on it.

Based on everything covered so far, abstraction as a guideline can be put into practice using several concrete tools in C#:

- **Interfaces** (e.g., `IShapeSeries`, `IComparable`, `ILogger`) - define a pure contract of what a type must be able to do, with zero implementation details exposed to the caller.
- **Abstract classes** (e.g., `GeometricShape` with an abstract `CalculateArea()` method) - define a partial abstraction: some behavior is fixed and shared, while specific behavior is deferred to each concrete subclass to implement.
- **Polymorphism** - writing code that operates on the abstract type (interface or base class) and lets the actual object's concrete type determine the real behavior at runtime, without the calling code needing to know which concrete type it's dealing with.

- **Encapsulation** - hiding a class's internal fields behind properties/methods, which is the mechanism that actually enforces the "hide how it works" half of abstraction.
- **The Factory pattern** - centralizing object creation (e.g., a ShapeFactory.CreateShape(...) method) so that calling code asks for "a shape of this type" without needing to know which concrete class gets constructed or how.

Put together, these tools let a program be built around stable, abstract contracts (interfaces and abstract classes) while the concrete, swappable details (SquareSeries vs CircleSeries, Rectangle vs Triangle, ConsoleLogger vs some other logger) stay hidden behind them - which is the practical, day-to-day expression of "abstraction" as a design guideline.